Operating Systems
Orange-2

SRN	PES1UG24CS599
Name	Preetham V
Class	4-J

Phase-1: Object Storage Foundation

Output of ./test_objects

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aa
fc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
```

Output showing sharded directory structrure

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase-2: Tree Objects

Output of ./test_tree

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Phase-3: The Index (Staging Area)

output of ./pes init, ./pes add file1.txt file2.txt, ./pes status

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % ./pes add file1.txt file2.txt
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  README.md
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.h
  untracked:  pes.c
  untracked:  pes.h
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  tree.h
  untracked:  Makefile
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  object.c
  untracked:  test_objects
  untracked:  test_tree
  untracked:  index.c
```

output of cat .pes/index

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776407524 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776407524 6 file2.txt
```

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
```

Phase-4: Commits and History

Required Screenshots

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
Initialized empty PES repository in .pes/
```

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % ./pes commit -m "Initial commit"
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
```

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
```

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
```

```
pre@fedora ~/projects/os_vcs/PES1UG24CS599-pes-vcs- % cat .pes/HEAD
ref: refs/heads/main
```

Phase-5 and 6: Analysis-Only Questions

Q5.1 — Implementing pes checkout <branch>

To implement checkout, the following steps are needed. First, read the target branch file from .pes/refs/heads/<branch> to get its latest commit hash. Second, update .pes/HEAD to point to the new branch (ref: refs/heads/<branch>). Third and most complex synchronize the working directory: walk the target commit's tree, extract each blob to its corresponding path, and delete files tracked in the current HEAD that are absent from the target tree. The complexity comes from conflict detection: if a file is dirty (modified in the working directory but not staged) and differs between the two branches, checkout must refuse and report the conflict rather than silently overwriting user work.

Q5.2 — Detecting Dirty Working Directory Conflicts

Conflicts are detected in two layers. First, compare the working file's mtime and size against the Index if they differ, the file has been modified since it was staged (it is unstaged-dirty). Second, compare the Index entry's hash against the current HEAD commit's tree hash for that file if they differ, the file is staged but not yet committed. If a file is dirty (by either test) and that same file exists with a different hash in the target branch's tree, checkout must abort with an error: "Your local changes would be overwritten by checkout." Files that are dirty but not present in the target branch are also blocked to prevent silent data loss

Q5.3 — Detached HEAD State

Detached HEAD means .pes/HEAD stores a raw commit hash instead of a symbolic branch reference. When new commits are made in this state, they are recorded in the object store and HEAD is updated to each new hash directly but no branch pointer advances with them. When the user switches branches or checks out a different commit, these orphaned commits become unreachable and will eventually be deleted by garbage collection. Recovery requires the user to note the commit hash from their terminal history, then run pes branch recover <hash> to create a new branch pointing to it before GC runs.

Q6.1 — Mark-and-Sweep GC Algorithm

The algorithm uses a classic mark-and-sweep approach. In the mark phase, we start from all branch tips by reading every file in .pes/refs/heads/ plus HEAD, then perform a BFS/DFS traversal: for each commit, add the commit hash, its tree hash, and all blob hashes reachable from that tree (recursively for subtrees) to a hash set (implemented as a hash table for O(1) lookup). Then walk each commit's parent pointer and repeat until the root. In the sweep phase, enumerate all files under .pes/objects/ and delete any whose hash is absent from the reachable set.

Q6.2 — GC Race Condition with Concurrent Commits

The race condition: (1) A commit operation writes a new blob object to the store but has not yet writtenthe tree or commit objects that reference it. (2) At this instant, GC scans the object store the blob exists but is unreferenced because the commit linking to it hasn't been written yet. (3) GC deletes the blob. (4) The commit operation continues, writes the tree pointing to the now-deleted blob, and updates HEAD the repository is now corrupt. Git avoids this with a 14-day grace period: GC only deletes objects whose filesystem mtime is older than two weeks. Any object written in the last 14 days is considered potentially in-flight and is preserved, regardless of whether it is currently reachable. This is a pragmatic tradeoff: it wastes a small amount of space for two weeks in exchange for complete safety against this race without requiring locks.